

Runtime Policy Enforcement at the AI Agent-to-Tool Boundary

github.com/poojakira/mcp-agent-security-gateway

Research Brief - Poster 01

Evidence status: Refreshed against code snapshot 8427f9ecafd3438a86775a7ceaf809f4ee051b5b and successful GitHub Actions CI run 36783059917 on 2026-09-30. The quantitative claims below are tied to that code snapshot. Do not generalize benchmark or test results beyond their stated scope.

Repository

github.com/poojakira/mcp-agent-security-gateway - public, default branch main.

Academic Project Title

Runtime Policy Enforcement at the AI Agent-to-Tool Boundary

Technical Subtitle

Design and Validation of an Inline MCP/JSON-RPC Security Gateway

One-Sentence Contribution

An application-layer inspection and authorization point for agent-generated MCP tools/call requests that combines JSON-RPC validation, normalization, trust/capability checks, heuristic content detection, policy evaluation, and tamper-evident audit telemetry before downstream tool execution.

Problem and Threat Model

LLM agents can turn untrusted natural-language context into structured tool calls with file, process, network, or privileged-service effects. The gateway protects the agent-to-tool boundary only when traffic is routed through it. Attackers may craft malicious tool arguments, malformed JSON-RPC, obfuscated prompt-injection strings, exfiltration payloads, or abusive request patterns.

Method

- Parse and validate JSON-RPC, including duplicate-key and malformed-input rejection.
- Normalize potentially obfuscated content before inspection.
- Evaluate server trust, capabilities, and content/policy signals.
- Return allow/block decisions using path-specific enforcement semantics.
- Record hash-chained audit evidence and SIEM-oriented telemetry.

Current Verified Evidence

GitLab Actions CI on Python 3.12 at 8427f9ecafd3438a86775a7ceaf809f4ee051b5b reports:

- 718 tests passed**, 0 failed.
- 82.46% statement coverage** - 5,524 statements, 969 missed.
- The Python 3.10 and 3.11 test jobs also completed successfully.
- Ruff/format, Pyright, Bandit, pip-audit, CodeQL, Windows control-plane validation, container build, Trivy/Grype/SBOM jobs completed successfully.
- 55** entries remain in the named prompt-injection pattern collection.
- 9** Elastic `[[rule]]` records.
- 21** core SIEM tests plus **12** SIEM scenario-runner tests.

Honest Boundaries

- No external population-level false-positive/false-negative rate is established.
- Synthetic correlation fixtures and local microbenchmarks are not production efficacy or SLO evidence.
- Enforcement semantics are not identical across every integration path.
- Only traffic routed through the gateway can be inspected or blocked.
- A successful CI suite is not evidence of a live production SOC deployment.

Reproducibility

```
git clone https://github.com/poojakira/mcp-agent-security-gateway.git

cd mcp-agent-security-gateway

git checkout 8427f9ecafd3438a86775a7ceaf809f4ee051b5b

python -m venv .venv

.\.venv\Scripts\Activate.ps1

python -m pip install -e ".[dev,server]"

$env:PYTHONPATH="src"

python -m pytest tests -q --cov=mcp_monitor --cov-report=term
```

Expected at the cited snapshot: **718 passed**, **82.46%** statement coverage.

Evidence Sources

- GitHub Actions CI run 36783059917
- VERIFIED_METRICS.md
- evidence/mcp_replay_evidence.json
- detection_rules/elastic_rules.toml
- tests/test_siem.py
- tests/test_siem_scenarios.py

References

Model Context Protocol specification; JSON-RPC 2.0; OWASP guidance for LLM applications; MITRE ATLAS; NIST AI RMF; Elastic Common Schema.